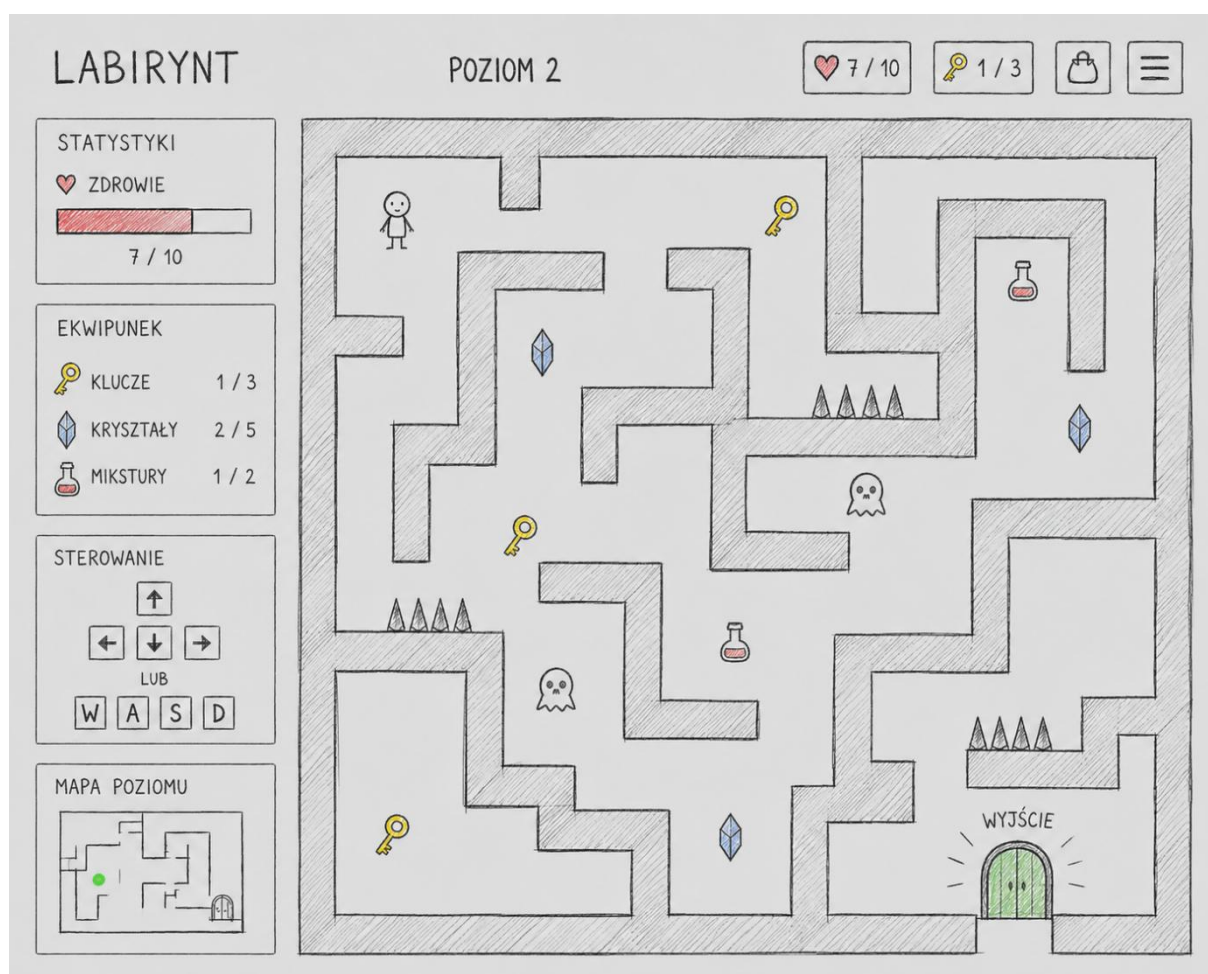


PROJEKT: LABIRYNT



Zespół programistyczny: Kacper Nowicki, Julia Malinowska, Michał Wysocki, Natalia Dąbek, Patryk Zieliński

Czas realizacji: 10.04.2026 – 08.06.2026

1. Ogólny opis projektu

1.1. Cel projektu

Celem projektu było stworzenie nowoczesnej gry przeglądarkowej typu **Labirynt** działającej w środowisku przeglądarki internetowej. Gracz steruje postacią przemieszczającą się po planszy, zbiera przedmioty, rozwiązuje zagadki logiczne oraz unika zagrożeń.

Gra została wykonana z wykorzystaniem technologii HTML5 Canvas oraz JavaScript. Projekt został opracowany zgodnie z założeniami metody Kanban.

1.2. Charakterystyka gry

Gra składa się z kilku poziomów o rosnącym poziomie trudności. Każdy poziom zawiera:

- labirynt,
- przeciwników,
- pułapki,
- przedmioty do zebrania,
- zagadki logiczne,
- punkt końcowy umożliwiający przejście dalej.

Gracz posiada określoną liczbę punktów życia (HP). Kontakt z zagrożeniem powoduje utratę zdrowia. Po utracie całego HP wyświetlany jest ekran końca gry.

2. Wymagania funkcjonalne

2.1. Sterowanie postacią

Opis funkcjonalności - gracz steruje postacią za pomocą klawiatury.

Funkcje:

- poruszanie klawiszami WASD,
- poruszanie strzałkami,
- blokada ruchu przez ściany,
- animacja ruchu,
- płynne przejścia pomiędzy polami.

2.2. Mechanika labiryntu

Funkcje:

- plansza oparta o siatkę (grid),
- ściany i przejścia,
- punkt startowy i końcowy,
- wykrywanie kolizji,
- przechodzenie między poziomami.

2.3. System przedmiotów

Dostępne przedmioty:

- klucze,
- bonusy HP,
- przedmioty fabularne,
- aktywatory zagadek.

Funkcjonalności:

- znikanie przedmiotów po podniesieniu,
- aktualizacja ekwipunku,
- aktualizacja punktów życia.

2.4. Zagadki logiczne

Typy zagadek:

- pytania logiczne,
- sekwencje liczb,
- kody dostępu,
- zagadki pamięciowe.

Funkcje:

- blokowanie przejścia do czasu rozwiązania,
- wyświetlanie komunikatów,
- sprawdzanie poprawności odpowiedzi.

2.5. System przeciwników

Funkcjonalności:

- ruch przeciwników po mapie,
- wykrywanie kolizji z graczem,
- utrata HP po kontakcie,
- pułapki i przeszkody.

2.6. System poziomów

Funkcje:

- minimum 3 poziomy,
- zwiększanie trudności,
- większe mapy,
- większa liczba przeciwników,
- trudniejsze zagadki.

2.7. Interfejs użytkownika

Elementy UI:

- licznik HP,
- aktualny poziom,
- ekwipunek,
- komunikaty systemowe,
- ekran zwycięstwa,
- ekran porażki.

2.8. Funkcja dodatkowe

- ekran startowy,
- możliwość restartu gry,
- efekty dźwiękowe,
- responsywność aplikacji.

3. Wymagania niefunkcjonalne

3.1. Wydajność

- płynne działanie gry,
- szybkie ładowanie poziomów,
- optymalizacja renderowania Canvas,
- lekkie zasoby graficzne.

3.2. Użyteczność

- prosty interfejs,
- czytelny układ planszy,
- intuicyjne sterowanie,
- wyraźne komunikaty.

3.3. Responsywność

Gra poprawnie działa na:

- komputerach stacjonarnych,
- laptopach,
- różnych rozdzielczościach ekranu.

3.4. Bezpieczeństwo

- walidacja danych wejściowych,
- zabezpieczenie przed błędami JavaScript,
- kontrola poprawności stanu gry.

4. Architektura projektu

4.1. Struktura katalogów

4.2. Opis modułów

- game.js - główna pętla gry, inicjalizacja poziomów oraz zarządzanie stanem gry
- player.js - obsługa gracza, ruchu, animacji oraz kolizji

SZKIC DOKUMENTACJI PROJEKTU

- enemy.js - sterowanie przeciwnikami oraz ich zachowaniami
- maze.js - tworzenie i obsługa mapy labiryntu
- items.js - obsługa przedmiotów i interakcji
- ui.js - wyświetlanie interfejsu użytkownika
- puzzles.js - logika zagadek logicznych

5. Dokumentacja kodu

5.1. Komentarze w kodzie

Kod źródłowy został opisany komentarzami w celu:

- zwiększenia czytelności,
- ułatwienia dalszego rozwoju,
- szybszego lokalizowania błędów.

Przykład:

```
// Funkcja odpowiedzialna za ruch gracza
function movePlayer(direction) {
  // aktualizacja pozycji
}
```

5.2. Opis kluczowych funkcji

- movePlayer() - obsługa ruchu gracza po planszy
- detectCollision() - wykrywanie kolizji ze ścianami oraz przeciwnikami
- loadLevel() - ładowanie nowego poziomu gry
- updateUI() - aktualizacja elementów interfejsu użytkownika
- startPuzzle() - uruchamianie zagadek logicznych

6. Zarządzanie projektem

6.1. Metodyka pracy

Projekt realizowany był zgodnie z metodyką Kanban. Narzędzia:

- Trello,
- GitHub,
- Visual Studio Code.

6.2. Podział ról

Osoba	Rola
Kacper Nowicki	Programista JavaScript
Julia Malinowska	Frontend / UI Designer
Michał Wysocki	Gameplay Programmer
Natalia Dąbek	Tester QA
Patryk Zieliński	GitHub / DevOps

7. Testowanie aplikacji

7.1. Testowanie funkcjonalne

Przeprowadzono testy:

- sterowania postacią,
- kolizji,
- działania przeciwników,
- poprawności poziomów,
- działania zagadek,
- poprawności UI.

7.2. Testowanie wydajności

Sprawdzono:

- płynność animacji,
- zużycie pamięci,
- szybkość ładowania poziomów.

8. Historia zmian i rozwój projektu

8.1. Wersjonowanie

Wersja	Opis
vo.1	Prototyp sterowania
vo.5	Dodanie labiryntów i kolizji
vo.8	Implementacja przeciwników
v1.0	Finalna wersja projektu

8.2. Lista zmian

- dodanie systemu poziomów,
- dodanie zagadek logicznych,
- poprawa interfejsu,
- optymalizacja renderowania,
- dodanie efektów dźwiękowych.

8.3. Możliwości dalszego rozwoju

- tryb multiplayer,
- generator losowych labiryntów,
- zapis stanu gry,
- ranking graczy,
- nowe typy przeciwników,
- rozbudowane AI.